iTCRL: Causal-Intervention-Based Trace Contrastive Representation Learning for Microservice Systems

Xiangbo Tian, Shi Ying, Tiangang Li, Mengting Yuan, Ruijin Wang, Yishi Zhao, and Jianga Shang, *Member, IEEE*

***Abstract*—Nowadays, microservice architecture has become mainstream way of cloud applications delivery. Distributed tracing is crucial to preserve the observability of microservice systems. However, existing trace representation approaches only concentrate on operations, relationships and metrics related to service invocations. They ignore service events that denotes meaningful, singular point in time during the service's duration. In this paper, we propose iTCRL, a novel trace contrastive representation learning approach based on causal intervention. This approach first constructs a unified graph representation for each trace to describe the runtime status of service events in traces and the complex relationships between them. Then, Causal-intervention-based Trace Contrastive Learning is proposed, which learns trace representations from causal perspective based on the unified graph representations of traces. It uses causal intervention to generate contrastive views, heterogeneous graph neural network-based trace encoder to learn trace representations, and direct causal effect to guide the training of trace encoder. Experimental results on three datasets show that iTCRL outperforms all baselines in terms of trace classification, trace anomaly detection, trace sampling and noise robustness, and also validate the contribution of Causal-intervention-based Trace Contrastive Learning.**

***Index Terms*—Microservice systems, distributed tracing, trace representation, causal intervention, graph contrastive learning.**

Manuscript received 31 December 2023; revised 30 July 2024; accepted 12 August 2024. Date of publication 20 August 2024; date of current version 17 October 2024. This work was supported in part by the National Key R & D Program of China under Grant 2022YFB3304300, in part by the National Natural Science Foundation Project of China under Grant 62072342 and Grant 61672392, and in part by the Fundamental Research Funds for the Central Universities under Grant 2042023kf1018. Recommended for acceptance by Á. Beszédes. *(Corresponding author: Shi Ying.)*

Xiangbo Tian, Shi Ying, Tiangang Li, and Mengting Yuan are with the School of Computer Science, Wuhan University, Wuhan 430072, China (e-mail: tianxiangbo@whu.edu.cn; yingshi@whu.edu.cn; tiangangli@whu.edu.cn; ymt@whu.edu.cn).

Ruijin Wang is with the School of Information and Software Engineering, University of Electronic Science and Technology of China, Chengdu 610054, China (e-mail: ruijinwang@uestc.edu.cn).

Yishi Zhao and Jianga Shang are with the School of Computer Science, China University of Geosciences, Wuhan 430074, China (e-mail: zhaoyishi@cug.edu.cn; jgshang@cug.edu.cn).

Digital Object Identifier 10.1109/TSE.2024.3446532

I. Introduction

With the rapid development of cloud computing, microservice architecture is applied by the increased cloud applications due to the advantages of modularity and elasticity [1], [2], [3], [4]. While it brings greater reliability and flexibility to software development, it brings new challenges to software operation and maintenance, such as higher dynamics and more complex dependencies. To address these challenges, the distributed tracing [5], [6] is widely used in microservice systems.

Microservice systems produce a large number of traces every day. These traces consist of much useful information, such as span events, span attributes, timestamps and span links. The information can help to improve the expressive ability of trace representations. However, existing trace representation approaches only concentrate on operations, relationships and metrics related to service invocations, but ignore service events denoting meaningful, singular time points during the service's duration.

In this paper, we propose iTCRL, a novel trace contrastive representation learning approach based on causal intervention. This approach first constructs a Service-Event-based Trace Graphs (SETGs) for each trace, using service events as nodes, the relationships between service events as edges and service metrics as node attributes or edge attributes. SETGs can effectively describe the runtime status of service events and the complex relationships between them. Then, Causal-intervention-based Trace Contrastive Learning is proposed, which learns trace representations from causal perspective based on SETGs. It uses causal intervention to generate contrastive views, heterogeneous graph neural network-based trace encoder to learn trace representations, and direct causal effect to guide the training of trace encoder. Experimental results on three datasets show that iTCRL outperforms all baselines in terms of trace classification, trace anomaly detection, trace sampling and noise robustness, and also validate the contribution of Causal-intervention-based Trace Contrastive Learning. In summary, our contributions are given as follows.

- To our best knowledge, we are the first to introduce service events that denote meaningful, singular point in time during the service's duration to help model traces.

0098-5589 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:31 UTC from IEEE Xplore. Restrictions apply.

- We design a Causal-intervention-based Trace Contrastive Learning approach, which can learn trace representation from causal perspective, and effectively avoid the creation of traces that do not exist in real world.
- We conduct comprehensive experiments on three datasets to show that iTCRL outperforms all baselines in terms of trace classification, trace anomaly detection, trace sampling and noise robustness.

The rest of the paper is organized as follows. Section II introduces the background of this research. Section III describes the specific details of iTCRL. Section IV gives the experimental results and analysis. Section V introduces the related works. Section VI gives conclusion, limitations and future work.

II. Background

Microservice systems are usually distributed systems, which replace the highly-coupled monolithic services with a set of loosely-coupled microservices. Although microservice architecture brings larger reliability and flexibility for software development, it brings new challenges to software operation and maintenance, such as more complex dependencies and larger dynamicity [7]. These challenges make distributed tracing become more and more important.

In OpenTelemetry specifications [8], spans are basic building blocks of traces and represent units of work or operation. In OpenTelemetry specifications, a span includes the following information.

- **Span Name**: It is a string that records the span name.
- **Parent Span ID**: It is a string that records the parent span of a span.
- **Start and End Timestamps**: They are timestamps that record the start and end of a span.
- **Span Context**: It is an immutable object that contains Trace ID, Span ID, Trace Flags and Trace State.
- **Span Attributes**: They are key-value pairs that records operation information.
- **Span Event**: It can be thought of as a structured log message (or annotation) on a span, typically used to denote a meaningful, singular point in time during the span's duration.
- **Span Status**: Each span has a status, containing 'Unset', 'Error' and 'OK'.

Based on **Span Attributes**, service invocation operation can be got. Based on **Span Events**, we can get the service events that denote meaningful, singular time points during the service's duration. Based on **Timestamps**, we can calculate many service metrics that describe services' execution status. Based on **Parent Span ID**, service invocation relationships can be got.

III. Approach

A. Definitions

Based on the OpenTelemetry specifications [8], some concepts are defined to better describe our proposed approach. These concepts are given as follows.

- **Service Event (SE)**. It is an event that denotes a meaningful, singular point in time during the service's duration.
- **Service Invocation Event (SIE)**. It is a type of SE that denotes the start and end of a span.
- **Service Runtime Event (SRE)**. It is a type of SE that denotes a meaningful, singular point in time during the span's duration.
- **Service Event Metric (SEM)**. It is the metric set that describes the status of SEs within a time interval.
- **Invocation Event Constant Template (IECT)**. It is the constant template of SIE.
- **Invocation Event Variable Template (IEVT)**. It is the variable template of SIE.
- **Runtime Event Constant Template (RECT)**. It is the constant template of SRE.
- **Runtime Event Variable Template (REVT)**. It is the variable template of SRE.
- **Service Event Runtime Metric (SERM)**. It is the metric set that describes the runtime status of SEs within a time interval.
- **Service Event Invocation Metric (SEIM)**. It is the metric set that describes the invocation status between SIEs within a time interval.

B. Overview of iTCRL

In this paper, we propose iTCRL, a novel trace contrastive representation learning approach based on causal intervention. This approach can learn trace representation from causal perspective based on SETGs. Fig. 1 shows its overview.

As shown in Fig. 1, iTCRL includes two components. **Service-Event-based Trace Graph Construction** constructs a Service-Event-based Trace Graph (SETG) for each trace. **Causal-intervention-based Trace Contrastive Learning** trains a trace representation learning model.

C. Service-Event-Based Trace Graph Construction

Service-Event-based Trace Graph Construction consists of four steps. **Trace Data Preprocessing** get SEMs, SIEs and SREs from trace data. **Span Data Parsing** extracts SERMs, SEIMs, IECTs, IEVTs, RECTs and REVTs by SEM Parsing, SIE Parsing and SRE Parsing. **Vector Representation** transforms SERMs, SEIMs, IECTs, IEVTs, RECTs and REVTs into vector representations. **Graph Construction** constructs a SETG for each trace based on the relationships between SEs and the above vector representations.

1) Trace Data Preprocessing: Based on the above definitions, the SEs of a trace are divided into SREs and SIEs. For the former, the structured log messages on a span are directly treated as SREs. For the latter, they are constructed by the API information in Span Attributes. Besides, 14 metrics are calculated as SEMs based on the span information to describe the operating environment status when traces are generated. Its role is similar to the response time in STV [9], but it is more fine-grained than the response time. When microservice systems are normal, it will not cause inaccurate trace representations. When microservice systems are abnormal, it can make trace representation better express the anomalies. Table I gives these metrics.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:31 UTC from IEEE Xplore. Restrictions apply.

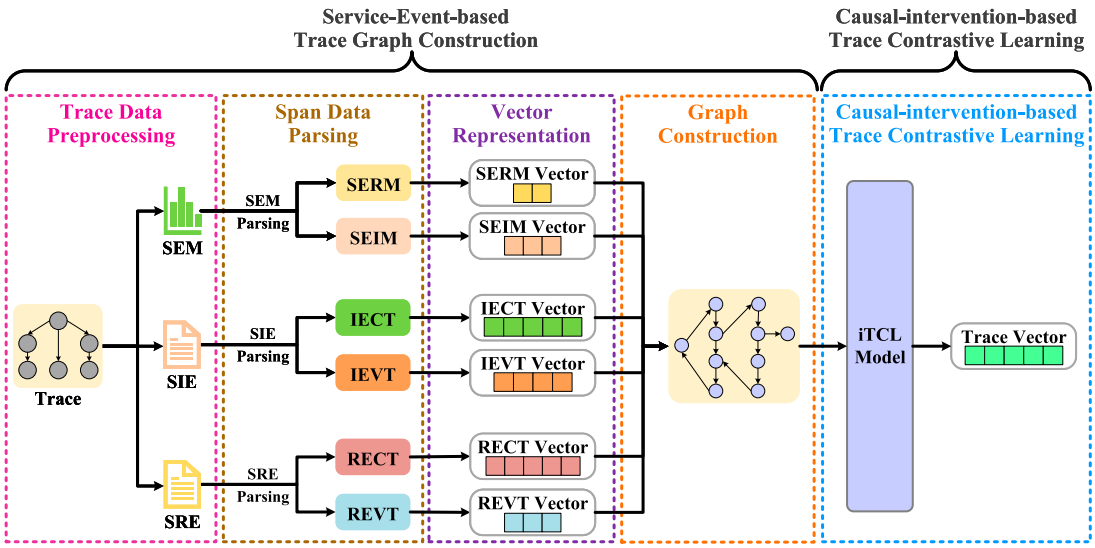


Fig. 1. Overview of iTCRL.

TABLE I
SERVICE EVENT METRICS

Metrics	Descriptions
Apdex	Application performance index.
CPM	The call number of a service instance per minute.
SLA	The call success rate of a service instance.
ART	The percentile of response time of a service instance, which contains P50, P75, P90, P95 and P99.
RTP	The percentile of response time of a service instance, which contains P50, P75, P90, P95 and P99.
CCPM	The call number of a service instance per minute detected on the client.
SCPM	The call number of a service instance per minute detected on the server.
CART	The average response time of a service instance detected on the client.
SART	The average response time of a service instance detected on the server.
CSLA	The call success rate of a service instance detected on the client.
SSLA	The call success rate of a service instance detected on the server.
NL	The network latency between service instances.
CRTP	The percentile of response time of a service instance detected on the client, which contains P50, P75, P90, P95 and P99.
SRTP	The percentile of response time of a service instance detected on the server, which contains P50, P75, P90, P95 and P99.

2) Span Data Parsing: **Span Data Parsing** includes three components: SEM Parsing, SIE Parsing and SRE Parsing.

SEM Parsing divides SEMs into SERMs and SEIMs according to their meanings and effects. The first five metrics in Table I are classified as SERMs, and the remaining metrics are classified as SEIMs.

SIE Parsing obtains IECTs and IEVTs from SIEs. Specifically, it extracts IECTs from SIEs by the Drain algorithm [10]. Then, it obtained IEVTs by comparing SIEs with its corresponding IECTs. For example, given a SIE 'INFO query 200' and its IECT 'INFO query $<*>$', its IEVT is '200'.

SRE Parsing obtains RECTs and REVTs from SREs. Its process is same as SIE Parsing.

3) Vector Representation: **Vector Representation** transforms SERMs, SEIMs, IECTs, IEVTs, RECTs and REVTs into vector representations.

For SERMs and SEIMs, they are directly transformed into vector representations, where each dimension represents a metric.

For IECTs, they are transformed into vector representations following the existing research [11], [12]. In detail, each IECT is first preprocessed, such as removing stop words, and splitting compound words. Then, a pre-trained GloVe model [13] is used to obtain vector representations of its each word. Finally, TF-IDF [14] is used to calculate its vector representation by aggregating the vector representations of its all words. For RECTs, their vector representation process is same as IECTs.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:31 UTC from IEEE Xplore. Restrictions apply.

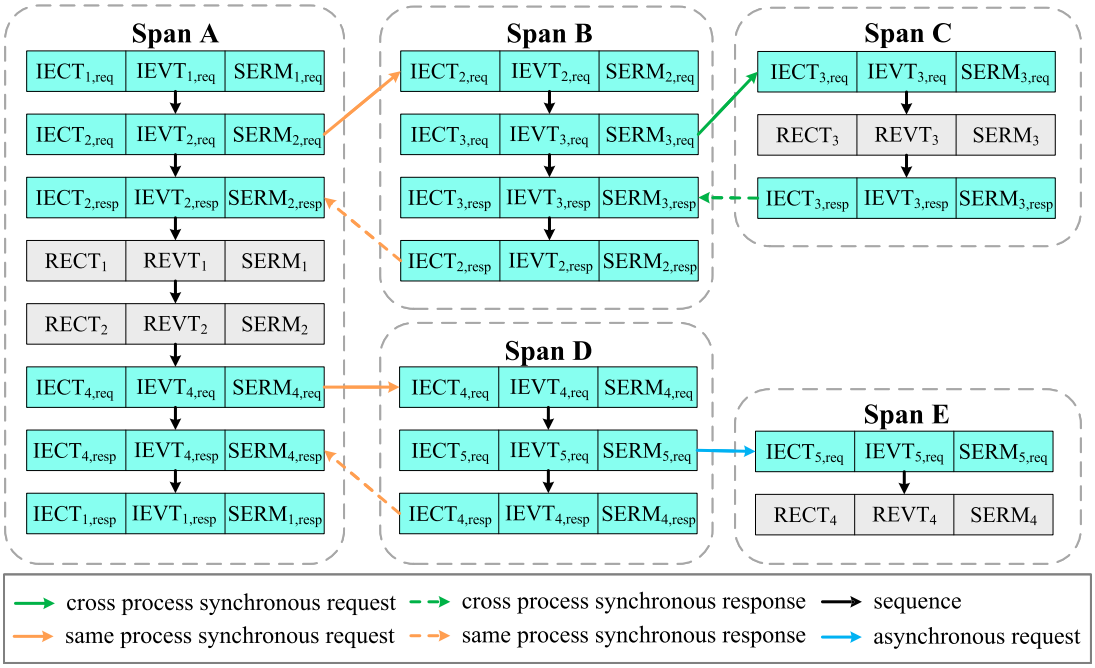


Fig. 2. An example of SETG.

For IEVTs, they are first divided into the word group and number group. Then, the same process as IECT vector representation is used to get vector representation of word group. Afterwards, the vector representation of number group is directly generated, where each dimension denotes a numerical variable. Finally, the vector representation of an IEVT is obtained by concatenating the two vector representations. For REVTs, their vector representation process is same as IEVTs.

4) Graph Construction: Graph-based data is powerful tools for describing complex relationships [15]. Therefore, each trace is first constructed as a SETG. In SETGs, nodes represent SEs that denote meaningful, singular points in time during the services' duration, which are described by IECT (RECT), IEVT (REVT) and SERM. Edges represent relationships between SEs, which are described by SEIM. The relationship construction consists of the connection between SEs in the same span and the connection between SEs in different spans. For the former, SEs in the same span are sequentially connected according to their timestamp. For the latter, we directly use span-linking method proposed in DeepTraLog [11]. Different to DeepTraLog, we refine synchronous request relationships into same process synchronous request relationships and cross process synchronous request relationships, and synchronous response relationships into same process synchronous response relationships and cross process synchronous response relationships.

Fig. 2 gives an example of SETG. In this graph, rectangles represent SEs and the elements in them represent the attributes of the corresponding SE. Dashed rounded rectangles indicate spans. Gray rectangles denote SREs and green rectangles denote SIEs. Different arrows indicate different types of relationships shown in the legend.

This graph well describes SEs in a trace and the relationships between them, which is extremely helpful for trace representations. In detail, the functional patterns of a trace can be well described by SEs and the relationships between them while its performance pattern can be well described by SERMs and SEIMs.

D. Causal-Intervention-Based Trace Contrastive Learning

Trace representation model needs to be trained with an unsupervised or self-supervised manner since the amount of trace data is large and the cost of labeling data is high. Graph contrastive learning [16], [17], [18] is an excellent self-supervised learning framework and very suitable for trace representation learning. However, it often creates traces that do not exist in real world when directly applied to trace representation learning. Therefore, Causal-intervention-based Trace Contrastive Learning (iTCL) approach is proposed, which learns trace representation from causal perspective. Fig. 3 gives the structure of iTCL. As shown in Fig. 3, this approach consists of Causal-intervention-based trace data augmentation (iTDA), Heterogeneous graph neural networks-based trace encoder (HTE) and Direct causal effect-based trace contrastive loss (DCE-TCL). Among them, iTDA defines three direct causes of traces and uses causal intervention to generate contrastive views, which can avoid the creation of traces that do not exist in real world. HTE gets trace representations by aggregating the structure information and attribute information on SETGs. DCE-TCL guides the training of HTE based on the direct causal effect between contrastive views.

1) Causal-Intervention-Based Trace Data Augmentation: Graph data augmentations are used to generate contrastive views, which are crucial for graph contrastive learning [17].

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:31 UTC from IEEE Xplore. Restrictions apply.

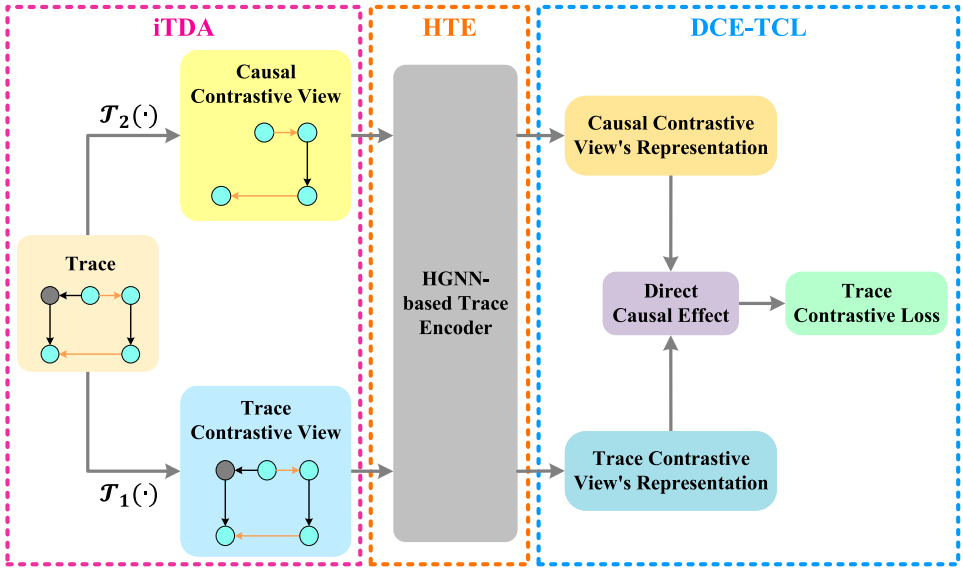


Fig. 3. Structure of iTCL.

There have been many different strategies, such as node dropping, edge perturbation, attribute masking and subgraph. Although these strategies are useful in general graph contrastive learning, some traces that do not exist in real world will be generated when they are directly applied to microservice trace representation learning. To solve this problem, Causal-intervention-based trace data augmentation (iTDA) is proposed.

According to the running mechanism of microservice systems, iTDA defines three direct causes of traces. The three direct causes are given as follows.

- **Service Invocation Status (SIS)**. It describes the invocation status between services.
- **Service Runtime Status (SRS)**. It describes the runtime status of services.
- **Internal and External Interaction Status (IEIS)**. It describes the interaction status inside and outside services.

To avoid the creation of traces that do not exist in real world, iTDA defines trace contrastive view and causal contrastive view, and design contrastive loss function based on them.

For trace contrastive views, the SETG of a trace is directly treated as its trace contrastive view. The specific generation process is given by the following equation.

$$\mathcal{T}_1(\mathcal{G}_i) = \mathcal{G}_i \tag{1}$$

where $\mathcal{G}_i$ denotes the SETG of trace i; $\mathcal{T}_1(\mathcal{G}_i)$ represents the trace contrastive view of trace i; $\mathcal{T}_1(\cdot)$ represents the generation strategy of trace contrastive views.

For causal contrastive views, causal intervention [19], [20], [21] is used to generate them based on traces' direct causes. The specific generation process is given by the following equation.

$$\mathcal{T}_2(\mathcal{G}_i, k) = do(\mathcal{G}_i = \mathcal{G}_i^k), k \in \{SIS, SRS, IEIS\} \tag{2}$$

where $\mathcal{T}_2(\mathcal{G}_i, k)$ denotes trace i's causal contrastive view; $\mathcal{T}_2(\cdot,\cdot)$ represents the generation strategy of causal contrastive views; $\mathcal{G}_i^k$ describes the SETG of trace i's direct cause k. The specific details of causal interventions are given as follows.

- $do(\mathcal{G}_i = \mathcal{G}_i^{SIS})$. It removes all SREs and the related edges from $\mathcal{G}_i$.
- $do(\mathcal{G}_i = \mathcal{G}_i^{SRS})$. It removes all SIEs and the related edges from $\mathcal{G}_i$.
- $do(\mathcal{G}_i = \mathcal{G}_i^{IEIS})$. It removes edges connecting same type of SEs and the resulting isolated nodes from $\mathcal{G}_i$.

Figs. 4–6 give the examples of three causal interventions for the trace in Fig. 2.

2) Heterogeneous Graph Neural Networks-Based Trace Encoder: For trace encoding task, we design HTE, a trace encoder based on heterogeneous graph neural networks. This approach encodes traces by aggregating the structure information and attribute information on SETGs. It consists of three components: node attribute aggregation, inter-neighbor aggregation, and trace pooling and projection.

a) Node Attribute Aggregation: In SETGs, a node is described by IECT (RECT), IEVT (REVT) and SERM. They lie in different feature spaces and have different feature dimensions [22]. Different dimensions of vector representations are troublesome when processing them in a unified framework. Therefore, an attribute-specific linear transformation is applied for each attribute vector representation of a node to project them into the same latent space before node attribute aggregation. For the attribute i of the node v, its latent vector representation $\mathbf{h}_{v,i}^t$ is calculated by the following equation:

$$\mathbf{h}_{v,i}^t = \mathbf{W}_i \mathbf{X}_{v,i}^t + \mathbf{b}_i \tag{3}$$

where t denotes the type of node v; $\mathbf{W}_i$ and $\mathbf{b}_i$ represent the parametric weight matrix and parametric bias matrix of attribute i, respectively; $\mathbf{X}_{v,i}^t$ describes the vector representation of attribute i of node v.

Next, node attribute aggregation is conducted. For a node, different attributes have different importance. Hence, the soft-attention mechanism is applied to describe the importance of different attributes. In detail, we learn a normalized importance

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:31 UTC from IEEE Xplore. Restrictions apply.

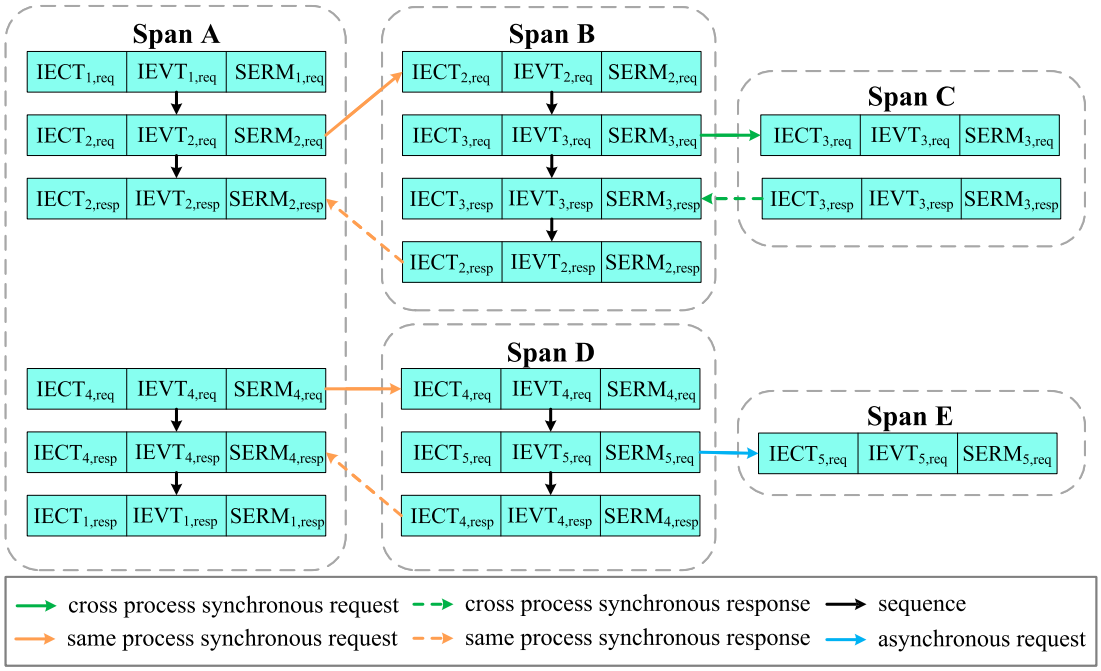


Fig. 4. An example of $do(\mathcal{G}_i = \mathcal{G}_i^{SIS})$.

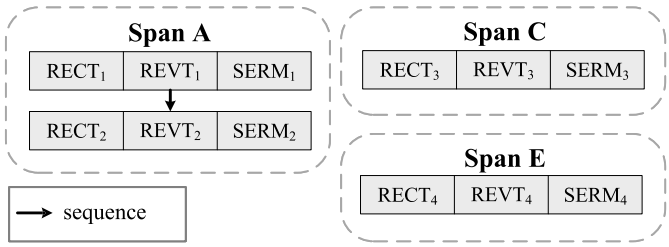


Fig. 5. An example of $do(\mathcal{G}_i = \mathcal{G}_i^{SRS})$.

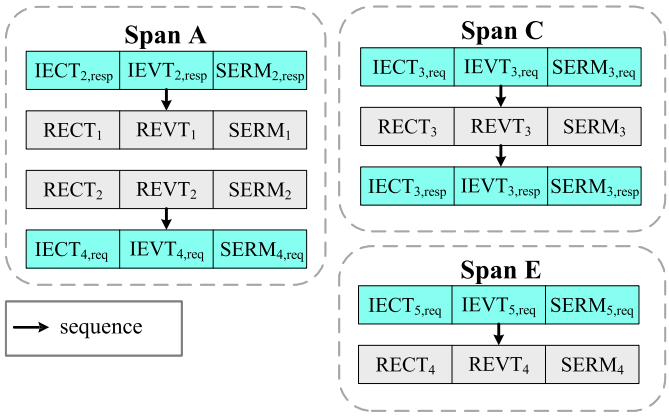


Fig. 6. An example of $do(\mathcal{G}_i = \mathcal{G}_i^{IEIS})$.

weight a_i for each attribute and then weighted sum all attributes to get nodes' vector representations. The attribute vector representation $\mathbf{h}_v^t$ of node v is calculated by the following equations:

$$\mathbf{o} = ReLU(\mathbf{W}_f[\mathbf{h}_{v,0}^t, \cdots, \mathbf{h}_{v,i}^t, \cdots, \mathbf{h}_{v,|\mathcal{A}|}^t] + \mathbf{b}_f) \quad (4)$$

$$a_i = \frac{e^{o_i}}{\sum_{j \in \mathcal{A}} e^{o_j}} \quad (5)$$

$$\mathbf{h}_v^t = tanh(\sum_{i \in \mathcal{A}} a_i \times \mathbf{h}_{v,i}^t) \quad (6)$$

where $\mathbf{o}$ denotes the attention coefficient matrix of node attributes; $\mathbf{W}_f$ and $\mathbf{b}_f$ are the parametric weight matrix and parametric bias matrix of attribute attention, respectively; o_i and a_i represent the attention coefficient and the normalized attention coefficient of attribute i, respectively; $\mathcal{A}$ represents the node attribute set.

b) Inter-Neighbor Aggregation: Inspired by HEAT [23], the inter-neighbor aggregation mechanism is designed. In SETGs, nodes represent service events and edges describe the relationships between them. They need to be considered separately in the inter-neighbor aggregation since they have different meanings.

For nodes, different types of nodes may lie in different feature space and have different feature dimensions [22]. Therefore, a type-specific linear transformation is applied for nodes to project their vector representations into the same latent space before inter-neighbor aggregation. For the node v, its latent vector representation $\mathbf{h}_v$ is calculated by the following equation:

$$\mathbf{h}_v = \mathbf{W}_t \mathbf{h}_v^t + \mathbf{b}_t \quad (7)$$

where $\mathbf{W}_t$ and $\mathbf{b}_t$ represent the parametric weight matrix and parametric bias matrix of type t; $\mathbf{h}_v^t$ is the attribute vector representation of node v.

For edges, their attributes and types have different meanings, and lie in different feature space [23]. Therefore, they need to be projected into the same latent space before the inter-neighbor aggregation. The latent type vector representation $\hat{\mathbf{e}}_{vu}^{type}$ and

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:31 UTC from IEEE Xplore. Restrictions apply.

latent attribute vector representation $\hat{\mathbf{e}}_{vu}^{attr}$ from node u to node v are calculated by the following equations:

$$\hat{\mathbf{e}}_{vu}^{type} = LeakyReLU(\mathbf{W}_{type}\mathbf{e}_{vu}^{type} + \mathbf{b}_{type}) \tag{8}$$

$$\hat{\mathbf{e}}_{vu}^{attr} = LeakyReLU(\mathbf{W}_{attr}\mathbf{e}_{vu}^{attr} + \mathbf{b}_{attr}) \tag{9}$$

where $\mathbf{W}_{type}$ and $\mathbf{b}_{type}$ are the parametric weight matrix and parametric bias matrix of edge type; $\mathbf{W}_{attr}$ and $\mathbf{b}_{attr}$ represent the parametric weight matrix and parametric bias matrix of edge attribute; $\mathbf{e}_{vu}^{type}$ denotes the vector representation of the edge type from node u to node v; $\mathbf{e}_{vu}^{attr}$ describes the attribute vector representation of the edge from node u to node v.

Next, the inter-neighbor aggregation is conducted. In this process, the soft-attention mechanism is used to describe the importance of different nodes. In detail, the aggregation message is first calculated based on node vector representations and edge vector representations. And then, we learn a normalized importance weight for each neighbor and weighted sum all neighbors' aggregation message. Besides, the residual is used to avoid overfitting. The specific process is given by the following equations:

$$\mathbf{M}_{vu} = \mathbf{W}_r[\mathbf{h}_u, \hat{\mathbf{e}}_{vu}^{attr}] + \mathbf{b}_r \tag{10}$$

$$o_{vu} = LeakyReLU\left(\mathbf{W}_A\left[\mathbf{h}_v, \mathbf{h}_u, \hat{\mathbf{e}}_{vu}^{type}, \hat{\mathbf{e}}_{vu}^{attr}\right] + \mathbf{b}_A\right) \tag{11}$$

$$a_{vu} = \frac{e^{o_{vu}}}{\sum_{p\in\varepsilon(v)} e^{o_{vp}}} \tag{12}$$

$$\mathbf{h}_v = \mathbf{h}_v + tanh(\sum_{u\in\varepsilon(v)} a_{vu} \times \mathbf{M}_{vu}) \tag{13}$$

where $\mathbf{M}_{vu}$ denotes the aggregation message from node u to node v; $\mathbf{W}_r$ and $\mathbf{b}_r$ are the parametric weight matrix and parametric bias matrix of inter-neighbor aggregation, respectively; $\mathbf{W}_A$ and $\mathbf{b}_A$ represent the parametric weight matrix and parametric bias matrix of neighbor attention, respectively; o_{vu} and a_{vu} are the attention coefficient and normalized attention coefficient from node u to node v, respectively; $\varepsilon(v)$ denotes the neighbor node set of node v; $\mathbf{h}_v$ is the vector representation of node v.

c) Trace Pooling and Projection: After inter-neighbor aggregation, trace representations need to be calculated based on the vector representations of all nodes in a trace. To reflect different contributions of nodes for a trace, the attention pooling is applied to calculate trace representations. The trace pooling result is calculated by the following equation:

$$\mathbf{H} = \sum_{v\in V} ReLU(\mathbf{W}_g[\mathbf{h}_v^t, \mathbf{h}_v] + \mathbf{b}_g) \odot ReLU(\mathbf{W}_p[\mathbf{h}_v^t, \mathbf{h}_v] + \mathbf{b}_p) \tag{14}$$

where $\mathbf{H}$ is the trace presentation; $\mathbf{W}_g$ and $\mathbf{b}_g$ are the parametric weight matrix and parametric bias matrix of node feature transformation, respectively; $\mathbf{W}_p$ and $\mathbf{b}_p$ are the parametric weight matrix and parametric bias matrix of node attention, respectively.

Finally, following previous research [16], [17], [18], iTCRL applies a two-layer perceptron to enhance the expression ability of trace representations.

3) Direct Causal Effect-Based Trace Contrastive Loss: Inspired by Moco [24], iTCL treats trace contrastive learning as a dictionary query task. Specifically, iTCL takes causal contrastive view as a query, its corresponding trace contrastive view as positive key and its irrelevant trace contrastive view as negative key. This method can effectively avoid the adverse effects of artificially defined contrastive views on trace representation learning. Trace contrastive views $\mathcal{T}_1(\mathcal{G}_i)$ and causal contrastive views $\mathcal{T}_2(\mathcal{G}_i, k)$ share the same trace encoder. The specific calculation process of their vector representations is given as follows.

$$\begin{cases} \mathbf{H}_i = & f(\mathcal{T}_1(\mathcal{G}_i)) \\ \hat{\mathbf{H}}_i^k = f(\mathcal{T}_2(\mathcal{G}_i, k)), & k \in \{SIS, SRS, IEIS\} \end{cases} \tag{15}$$

where $f(\cdot)$ represents the trace encoder based on heterogeneous graph neural networks; $\mathbf{H}_i$ and $\hat{\mathbf{H}}_i^k$ are the vector representation of $\mathcal{T}_1(\mathcal{G}_i)$ and $\mathcal{T}_2(\mathcal{G}_i, k)$, respectively.

Based on the direct causal effect between causal contrastive view and trace contrastive view, Direct causal effect-based trace contrastive loss (DCE-TCL) is designed as follows.

$$\mathcal{L} = -\frac{1}{m}\log\sum_{i=1}^{m}\frac{\exp(DCE(\hat{\mathbf{H}}_i^k, \mathbf{H}_i)/\tau)}{\sum_{j=1}^{m}\exp(DCE(\hat{\mathbf{H}}_i^k, \mathbf{H}_j)/\tau)} \tag{16}$$

where $\mathcal{L}$ denotes the trace contrastive loss; τ is the temperature parameter; m represents the number of samples in a mini-batch; $DCE(\hat{\mathbf{H}}_i^k, \mathbf{H}_i)$ represents the direct causal effect between $\hat{\mathbf{H}}_i^k$ and $\mathbf{H}_i$.

As shown in Equation 14 and 15, $\mathbf{H}_i$ is the attention pooling result of SEs belonging to $\mathcal{T}_1(\mathcal{G}_i)$ while $\hat{\mathbf{H}}_i^k$ is the attention pooling result of SEs belonging to $\mathcal{T}_2(\mathcal{G}_i, k)$. According to Equation 1 and 2, $\mathcal{T}_2(\mathcal{G}_i, k)$ is a subgraph of $\mathcal{T}_1(\mathcal{G}_i)$. Therefore, there is a linear causal relationship between $\mathbf{H}_i$ and $\hat{\mathbf{H}}_i^k$, which is given as follows.

$$\mathbf{H}_i = \sum_{k\in\{SIS,SRS,IEIS\}} \alpha_k \times \hat{\mathbf{H}}_i^k \tag{17}$$

where α_k denotes the direct causal effect parameter between $\mathbf{H}_i$ and $\hat{\mathbf{H}}_i^k$. According to Equation 17, the direct causal effect parameter α_k can be calculated as follows.

$$\alpha_k = \frac{\|\mathbf{H}_i\| \times \|\hat{\mathbf{H}}_i^k\|}{\hat{\mathbf{H}}_i^k \cdot \hat{\mathbf{H}}_i^k} \times \cos(\mathbf{H}_i, \hat{\mathbf{H}}_i^k) - \frac{\sum_{l\in\{SIS,SRS,IEIS\}\wedge l\neq k} \alpha_l \times \hat{\mathbf{H}}_i^l \cdot \hat{\mathbf{H}}_i^k}{\hat{\mathbf{H}}_i^k \cdot \hat{\mathbf{H}}_i^k} \tag{18}$$

As shown in Equation 18, there is a linear relationship between the direct causal effect parameter α_k between $\mathbf{H}_i$ and $\hat{\mathbf{H}}_i^k$ and the cosine similarity between them. Therefore, the cosine-similarity is used to describe the direct causal effect. Equation 16 is modified as follows.

$$\mathcal{L} = -\frac{1}{m}\log\sum_{i=1}^{m}\frac{\exp(\cos(\hat{\mathbf{H}}_i^k, \mathbf{H}_i)/\tau)}{\sum_{j=1}^{m}\exp(\cos(\hat{\mathbf{H}}_i^k, \mathbf{H}_j)/\tau)} \tag{19}$$

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:31 UTC from IEEE Xplore. Restrictions apply.

As shown in Equation 19, there is a large overlap of positive sample pairs and negative sample pairs since different traces are likely to have same causal contrastive views. This problem leads to conflicting optimization direction. To solve this problem, Equation 19 needs to be adjusted. As shown in Equation 17, $\mathbf{H}_j$ is the weighted sum of all $\hat{\mathbf{H}}_j^k$. Therefore, the direct causal effect minimization between $\hat{\mathbf{H}}_i^k$ and $\mathbf{H}_j$ can be indirectly ensured by minimizing the direct causal effect between $\hat{\mathbf{H}}_i^k$ and $\hat{\mathbf{H}}_j^l$. Equation 19 is adjusted as follows.

$$\mathcal{L} = -\frac{1}{m}\log\sum_{i=1}^{m}\frac{\exp(\cos(\hat{\mathbf{H}}_i^k, \mathbf{H}_i)/\tau)}{\sum_{j=1}^{m}\exp(\cos(\hat{\mathbf{H}}_i^k, \hat{\mathbf{H}}_j^l)/\tau)} \tag{20}$$

IV. Evaluation

iTCRL is implemented using Python 3.7, PyTorch 1.11.0 and PyTorch Geometric 2.1.0. Based on this implementation, the effectiveness of the proposed approach is evaluated with the following three tasks:

- **Trace Classification**. Trace Classification aims to accurately identify traces according to trace representations, which benefits various downstream trace analysis tasks.
- **Trace Anomaly Detection**. Trace anomaly detection aims to accurately detect anomaly of microservice systems according to trace representations.
- **Trace Sampling**. Trace sampling aim to collect traces as few as possible while preserving system observability according to trace representations.

By the three tasks, the following research questions need to be answered:

- **RQ1**: How much does hyperparameters influence the effectiveness of iTCRL?
- **RQ2**: How effective is iTCRL in trace classification compared with baselines?
- **RQ3**: How effective is iTCRL in trace anomaly detection compared with baselines?
- **RQ4**: How effective is iTCRL in trace sampling compared with baselines?
- **RQ5**: How robust is iTCRL in three evaluation tasks compared with baselines?
- **RQ6**: How much does iTCL contribute to the effectiveness of iTCRL compared with traditional graph contrastive learning approaches?

A. Experimental Setup

1) Datasets: Experiments were conducted on three datasets collected from Train Ticket microservice system, the Social Network microservice system and the Online Boutique microservice system. These three microservice systems have been widely used in previous studies [11], [25], [26], [27], [28]. Among them, the Train Ticket microservice system has complex service dependencies, which makes service runtime events more important for trace analysis. The last two microservice systems have simple service dependencies, which makes service invocation events more important for trace analysis. The details of the datasets are shown in Table II.

TABLE II
Details of Datasets

Datasets	Instances	Normal	Anomalous	Classes
TT	45	151911	15138	57
SN	21	13612	1106	10
OB	11	56920	11882	12

- **TrainTicket (TT)**. This dataset is collected from the Train Ticket microservice system [11], [27], [28] composed of 45 microservices, which is currently the largest and most complex open source microservice system with our knowledge. Based on this system, we implement a normal version and five fault versions (including Monolithic Fault, Configuration Fault, CPU Exhaustion Fault, Network Fault, and Mixture Fault of CPU and Network), and deployed them on a Kubernetes cluster composed of 6 machines with Intel Core i7-4790 3.60GHz CPU and 8GB RAM. 151911 normal traces and 15138 anomalous traces are collected. According to the root operation and span number of traces, traces are divided into 57 classes. For trace contrastive learning, trace classification and trace sampling, the normal traces are divided into the training, validation and testing sets of $121529(80\%)$, $15191(10\%)$ and $15191(10\%)$, respectively. For trace anomaly detection task, all anomalous traces are put into testing set.
- **SocialNetwork (SN)** [25]. This dataset provided by [25]. It is collected from the Social Network microservice system [29], which is a broadcast-style social network site composed of 21 microservices. In this experiments, 13612 normal traces and 1106 anomalous traces of this dataset are used. According to the root operation and span number of traces, traces are divided into 10 classes. For trace contrastive learning, trace classification and trace sampling, the normal traces are divided into the training, validation and testing sets of $10890(80.00\%)$, $1361(10.00\%)$ and $1361(10.00\%)$, respectively. For trace anomaly detection task, all anomalous traces are put into testing set.
- **OnlineBoutique (OB)** [26]. This dataset provided by [26]. It is collected from the Online Boutique microservice system [30], which is an e-commerce microservice system composed of 11 microservices. In this experiments, 56920 normal traces and 11882 anomalous traces of this dataset are used. According to the root operation and span number of traces, traces are divided into 12 classes. For trace contrastive learning, trace classification and trace sampling, the normal traces are divided into the training, validation and testing sets of $45536(80.00\%)$, $5692(10.00\%)$ and $5692(10.00\%)$, respectively. For trace anomaly detection task, all anomalous traces are put into testing set.

2) Evaluation Metrics: For trace classification task, Macro-F1 and Micro-F1 are used to evaluate its effectiveness. For trace anomaly detection task, F1-Score is used to evaluate its effectiveness. For trace sampling task, Coverage [31] and Entropy

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:31 UTC from IEEE Xplore. Restrictions apply.

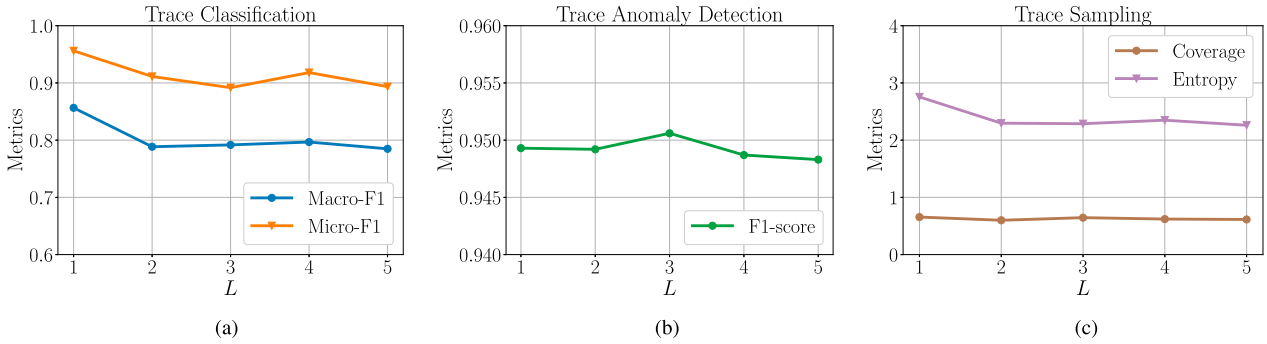


Fig. 7. Impact of hyperparameter L on TT dataset.

[31] are used to evaluate its effectiveness. For the impact of hyperparameters, all evaluation tasks need to be comprehensively considered. Therefore, an impact score function is defined based on all evaluation metrics, which is given as follows.

$$Score(i) = \frac{1}{C}\sum_{j=1}^{C} m_{i,j} \times \frac{1}{C}\sum_{j=1}^{C} \frac{m_{i,j} - \min_k m_{k,j}}{\max_k m_{k,j} - \min_k m_{k,j}} \tag{21}$$

where i is the hyperparameter value; j denotes an evaluation metric; C represents the number of evaluation metrics; k is the number of hyperparameter values.

3) Settings: To ensure the fairness of experiments, all experiments were conducted on a server with Intel Core i7-12700 2.10GHz CPU, 64GB RAM, RTX 4080 with 16GB GPU memory.

4) Baselines: To evaluate the effectiveness of iTCRL, four trace representation approaches are selected as baselines.

- **Operation Count Vector (OCV)** [32]. A rule-based trace representation approach. This approach represents traces based on service invocation operations, where each dimension represents a service invocation operation and its value is the invocation number of the corresponding operation.
- **Service Trace Vector (STV)** [9]. A rule-based trace representation approach. This approach represents traces based on service invocation paths and response time, where each dimension denotes a service invocation path and its value is the corresponding response time.
- **TraceCRL** [27]. A GNN-based trace representation approach. This approach combines graph neural network with contrastive learning to learn trace representations.
- **STEAM** [31]. A GNN-based trace representation approach. This approach incorporates graph neural network with domain knowledge of trace comparison through logical clauses to learn trace representations.

To verify the contribution of iTCL, we derive three variants of iTCRL by replacing iTCL with GraphCL [17], JOAO [18] and AutoGCL [16].

- **SETG-HTE-GraphCL**. It is derived by replacing the iTCL of iTCRL with GraphCL.
- **SETG-HTE-JOAO**. It is derived by replacing the iTCL of iTCRL with JOAO.
- **SETG-HTE-AutoGCL**. It is derived by replacing the iTCL of iTCRL with AutoGCL.

For the above baselines, GraphCL, JOAO and AutoGCL provide open-source implementations. Therefore, we directly use them. For other baselines, we develop them based on their papers.

B. RQ1: Impact of Hyperparameters

iTCRL has four important hyperparameters: the hidden layer number L, the temperature parameter τ, the embedding dimension D and the learning rate η. They determine the iterations of inter-neighbor aggregation on SETGs, the attention for causal contrastive views, the size of trace representation vectors and the convergence status of trace representation models, respectively.

Figs. 7–10 show the impact of hyperparameter L on iTCRL when other hyperparameters are fixed. As show in Figs. 7–9, the effectiveness of trace classification drops with L, and the effectiveness of trace anomaly detection first rises and then drops with L. The effectiveness of trace sampling drops with L on TT dataset and retains stable on other datasets. The reason is that bigger L leads iTCRL to over-smoothing, but the information loss in some extent leads abnormal traces to deviate further from the normal pattern, which benefits trace anomaly detection. As shown in Fig. 10, the best comprehensive performance is achieved at $L = 1$ on the three datasets.

Figs. 11–14 show the impact of hyperparameter τ on iTCRL when other hyperparameters are fixed. As show in Figs. 11–13, the effectiveness of trace classification decreases with τ. The effectiveness of trace anomaly detection first rises and then drops with τ on OB dataset while dropping with τ on other datasets. The effectiveness of trace sampling rises with τ on TT dataset, and retains stable on other datasets. The reason is that bigger τ leads iTCRL to allocate less attention for direct causes difficult to learn, which makes trace representations indistinguishable. As shown in Fig. 14, the best comprehensive performance is achieved at $\tau = 0.05$ on the three datasets.

Figs. 15–18 show the impact of hyperparameter D on iTCRL when other hyperparameters are fixed. As show in Figs. 15–17, the effectiveness of trace classification first rises and then drops with D on TT dataset and SN dataset while it first drops and

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:31 UTC from IEEE Xplore. Restrictions apply.

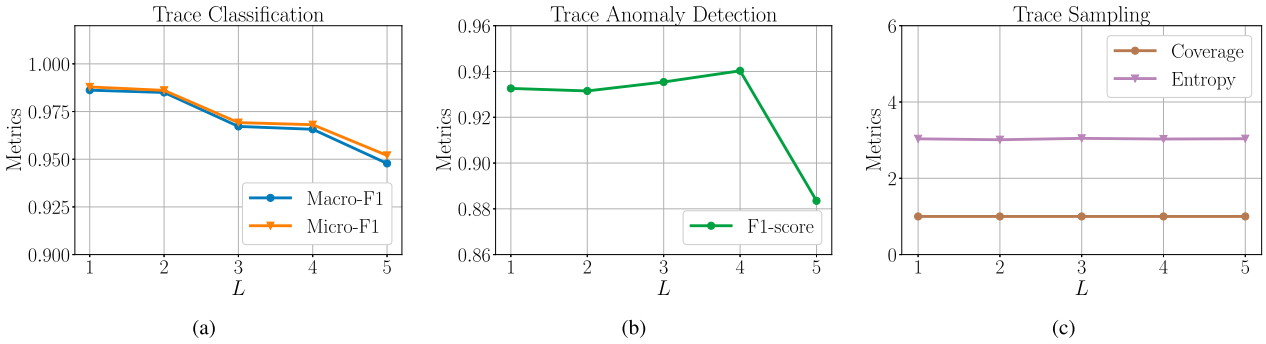


Fig. 8. Impact of hyperparameter L on SN dataset.

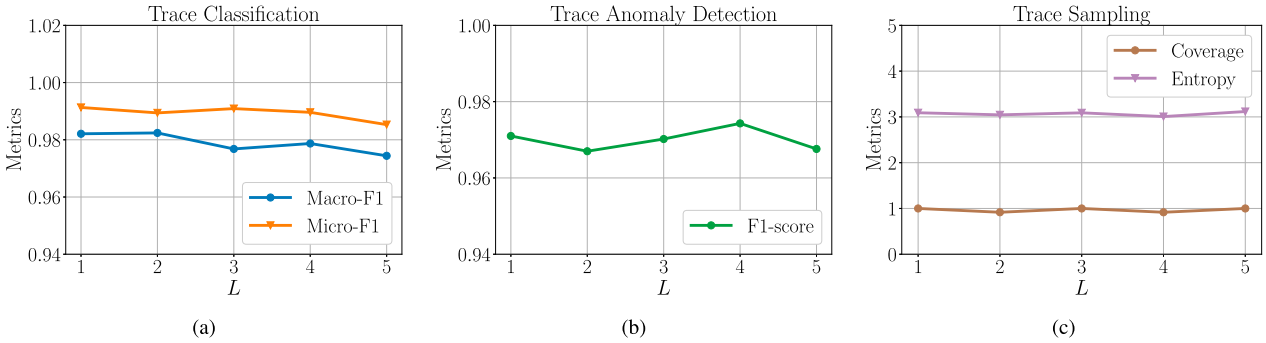


Fig. 9. Impact of hyperparameter L on OB dataset.

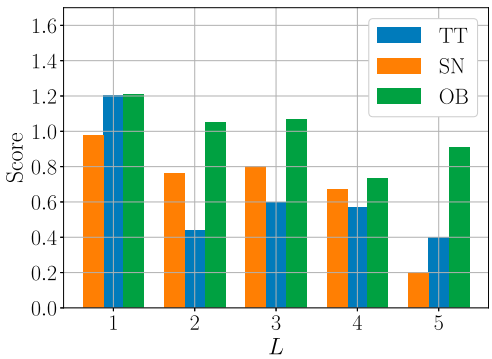


Fig. 10. Comprehensive Impact of hyperparameter L on three datasets.

then rises on OB dataset. The effectiveness of trace anomaly detection firsts drops and then rises with D on OB dataset while rising with D on other datasets. The effectiveness of trace sampling retains stable on OB dataset, and rises with D on other datasets. The reason is that bigger representation dimension leads model to overfitting, but the overfitting in some extent makes the model more sensitive to unseen traces, which helps trace anomaly detection. As shown in Fig. 18, the best comprehensive performance is achieved at $D = 64, 32, 64$ on TT, SN and OB dataset, respectively.

Figs. 19–22 show the impact of hyperparameter η on iTCRL when other hyperparameters are fixed. As show in Figs. 19–21, the performance of three evaluation tasks changes differently with η on different datasets and their best performance is achieved at different η. As shown in Fig. 22, the best comprehensive performance is achieved at $\eta = 10^{-4}, 10^{-5}, 10^{-4}$ on TT, SN and OB dataset, respectively.

In summary, the best configuration of iTCRL is given in Table III.

C. RQ2: Effectiveness for Trace Classification

To evaluate the effectiveness of iTCRL in terms of trace classification, we provide the trace representations generated by each trace representation approach for the SVM classifier. To minimize the impact of randomness, the SVM classifier is run 10 times independently and report their average. The experimental results are given in Table IV.

As shown in Table IV, iTCRL outperforms all baselines on TT dataset. This is because it considers service events that denotes meaningful, singular point in time during the service's duration, which makes trace representation more distinguishable. On SN dataset and OB dataset, it is slightly worse than the best baseline, ranking second and fourth respectively. The reason is that the scale of the Social Network microservice system and the Online Boutique microservice system is so small that their traces can be distinguished by root operation alone, and the less the additional information, the better trace classification. Comprehensively considering the effectiveness of all approaches on the three datasets, iTCRL is about 3.28%

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:31 UTC from IEEE Xplore. Restrictions apply.

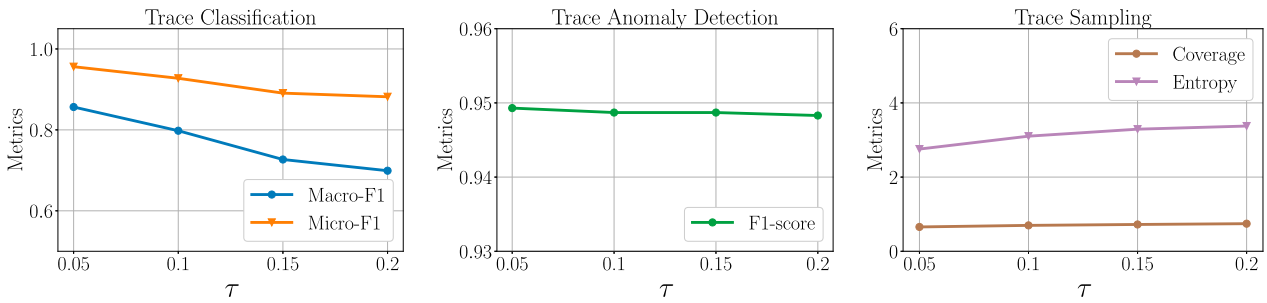


Fig. 11. Impact of hyperparameter τ on TT dataset.

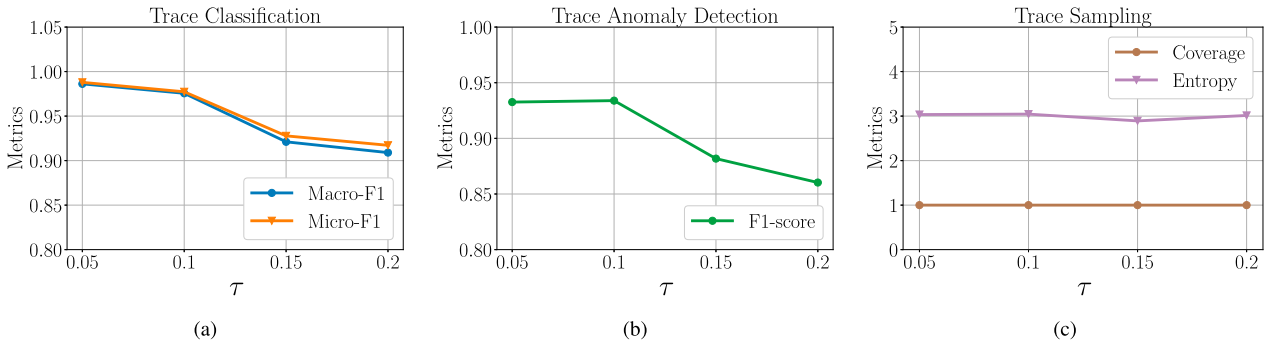


Fig. 12. Impact of hyperparameter τ on SN dataset.

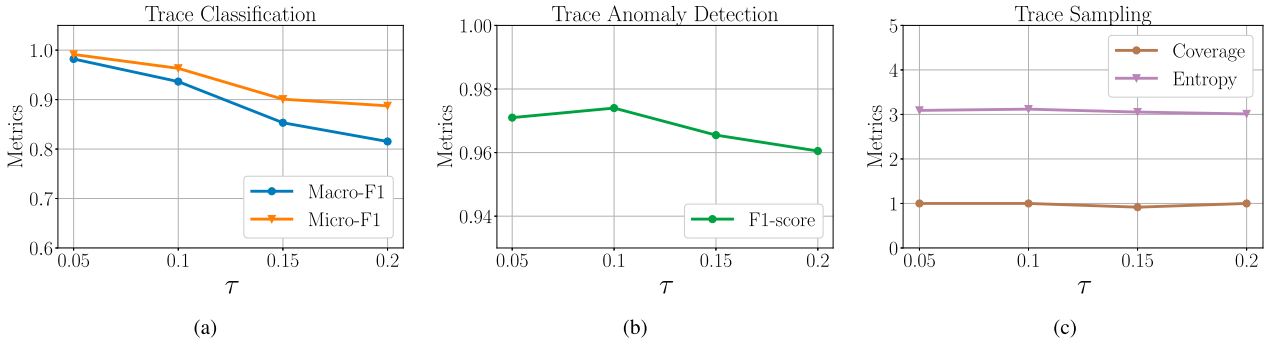


Fig. 13. Impact of hyperparameter τ on OB dataset.

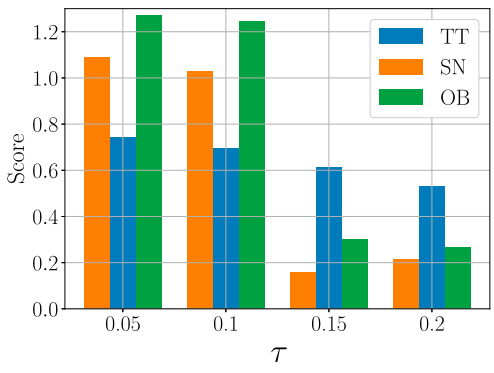


Fig. 14. Comprehensive Impact of hyperparameter τ on three datasets.

and 1.19% better than the best baseline (STEAM) in terms of Macro-F1 and Micro-F1, respectively.

D. RQ3: Effectiveness for Trace Anomaly Detection

To evaluate the effectiveness of iTCRL in terms of trace anomaly detection, we provide the trace representations generated by each trace representation approach for the LOF algorithm. To minimize the impact of randomness, the LOF algorithm is repeated 10 times and report their average. Table V shows the effectiveness evaluation results of trace anomaly detection.

As shown in Table V, iTCRL outperforms all baselines on three datasets. In detail, it is about 24.62%, 18.73% and 36.7%

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:31 UTC from IEEE Xplore. Restrictions apply.

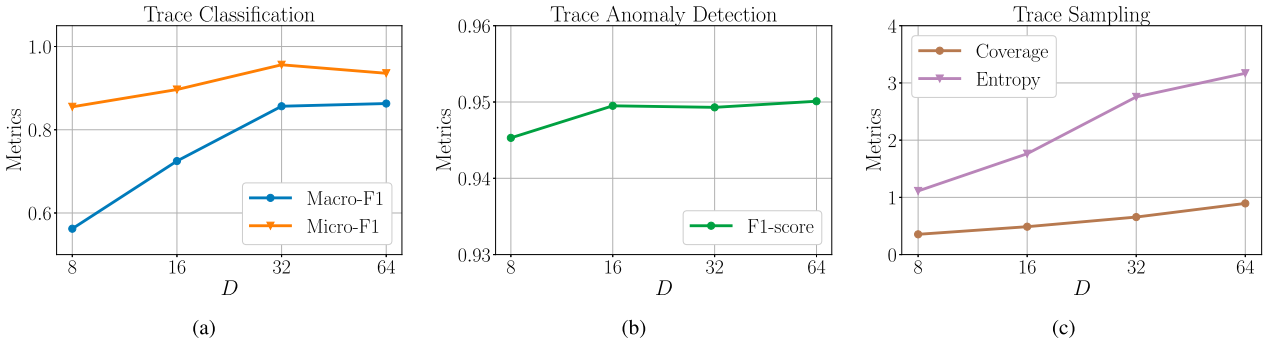


Fig. 15. Impact of hyperparameter D on TT dataset.

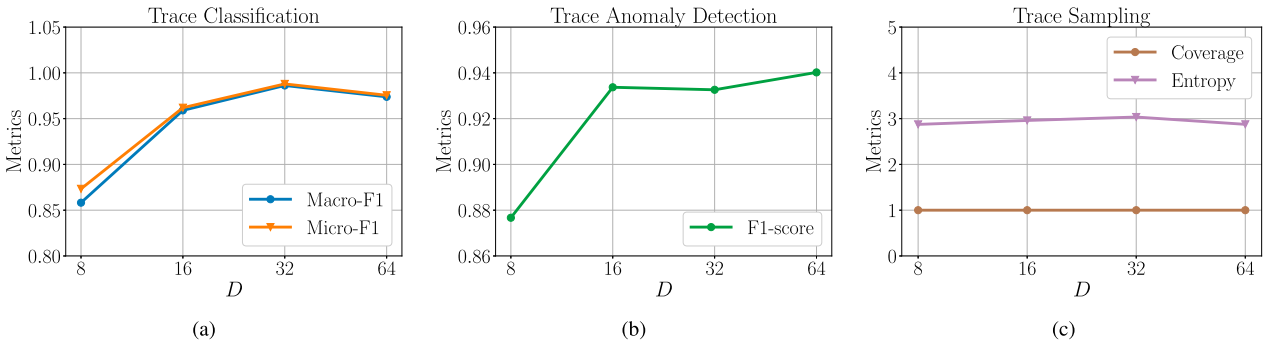


Fig. 16. Impact of hyperparameter D on SN dataset.

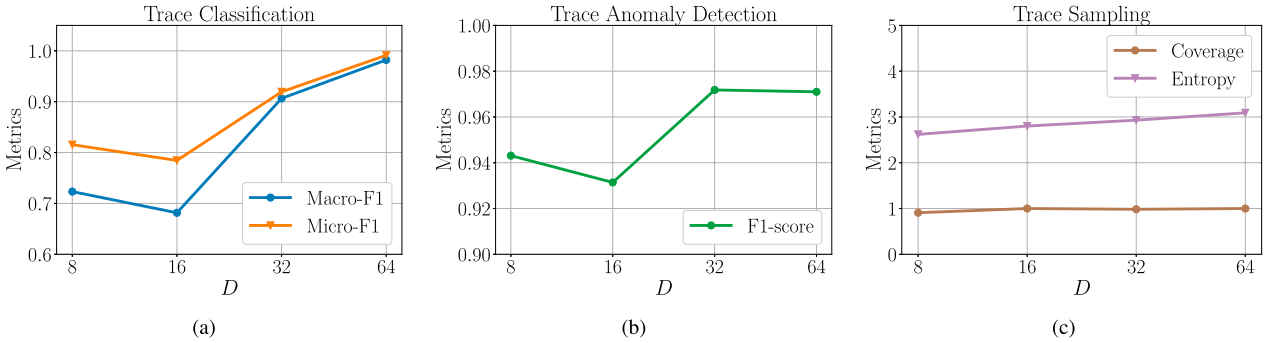


Fig. 17. Impact of hyperparameter D on OB dataset.

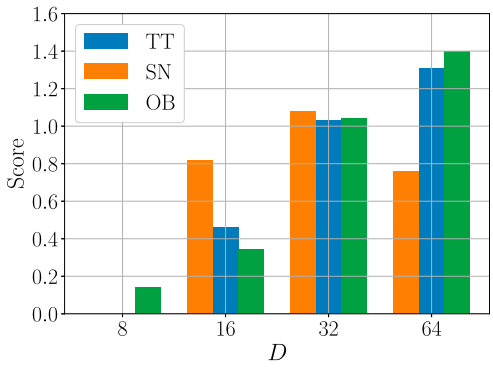


Fig. 18. Comprehensive Impact of hyperparameter D on three datasets.

better than the best baseline (STV, STV and STEAM) on TT dataset, SN dataset and OB dataset, respectively. The improvement is because iTCRL considers service events that denotes meaningful, singular point in time during the service's duration, their related metrics and the relationships between service events, which allows trace representation to reflect a variety of anomalies such as CPU exhaustion, memory exhaustion and network delays, etc. Comprehensively consider the effectiveness of all approaches on the three datasets, iTCRL is about 31.11% better than the best baseline (STV).

E. RQ4: Effectiveness for Trace Sampling

To evaluate the effectiveness of iTCRL in terms of trace sampling, we feed the trace representations generated by each trace

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:31 UTC from IEEE Xplore. Restrictions apply.

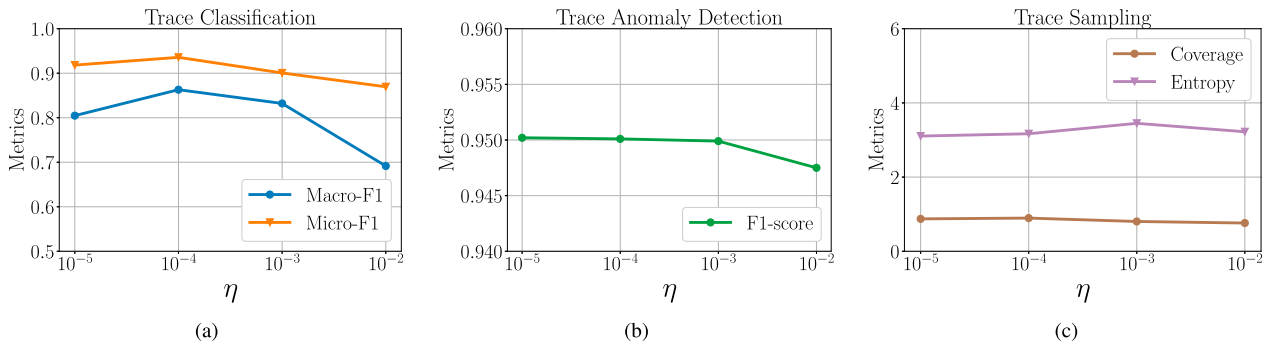


Fig. 19. Impact of hyperparameter η on TT dataset.

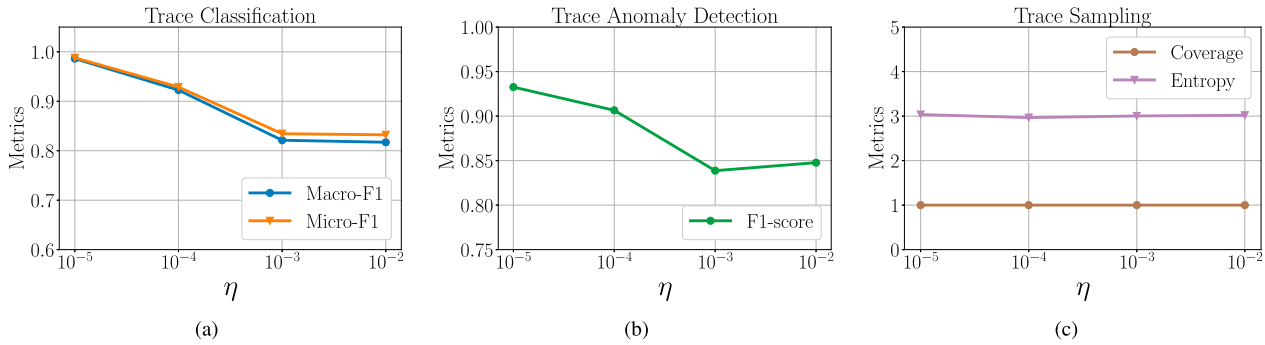


Fig. 20. Impact of hyperparameter η on SN dataset.

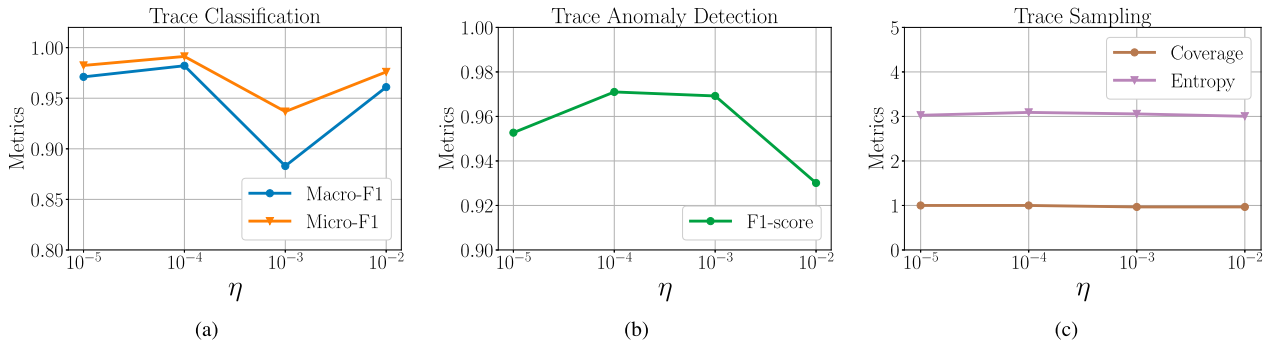


Fig. 21. Impact of hyperparameter η on OB dataset.

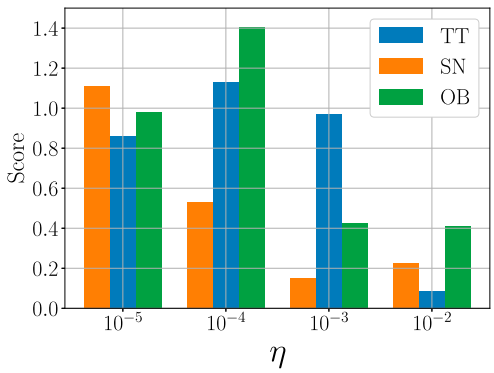


Fig. 22. Comprehensive Impact of hyperparameter η on three datasets.

TABLE III
CONFIGURATION OF iTCRL

Dataset	L	τ	D	η
TT	1	0.05	64	10^{-4}
SN	1	0.05	32	10^{-5}
OB	1	0.05	64	10^{-4}

representation approach to the DPP [31] algorithm and sample 1% traces from datasets. To minimize the impact of randomness, the DPP algorithm is repeated 10 times and report their average. Table VI shows the effectiveness evaluation results of trace sampling. As shown in Table VI, iTCRL outperforms all

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:31 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
EFFECTIVENESS EVALUATION RESULTS OF TRACE CLASSIFICATION

Datasets	Metrics	OCV	STV	TraceCRL	STEAM	iTCRL
TT	Macro-F1	0.7499	0.7216	0.4540	0.7919	**0.8630**
	Micro-F1	0.7315	0.7141	0.6434	0.9215	**0.9357**
SN	Macro-F1	**1.0000**	0.5444	0.8161	0.9504	0.9862
	Micro-F1	**1.0000**	0.6608	0.7916	0.9596	0.9879
OB	Macro-F1	**1.0000**	0.9976	0.9864	0.9992	0.9821
	Micro-F1	**1.0000**	0.9984	0.9802	0.9994	0.9913
Average	Macro-F1	0.9166	0.7545	0.7522	0.9138	**0.9438**
	Micro-F1	0.9105	0.7911	0.8051	0.9602	**0.9716**

TABLE V
EFFECTIVENESS EVALUATION RESULTS OF TRACE ANOMALY DETECTION

Datasets	OCV	STV	TraceCRL	STEAM	iTCRL
TT	0.2594	0.7624	0.4156	0.5894	**0.9501**
SN	0.1098	0.7855	0.3238	0.5387	**0.9326**
OB	0.1692	0.6286	0.3966	0.7103	**0.9710**
Average	0.1795	0.7255	0.3787	0.6128	**0.9512**

TABLE VI
EFFECTIVENESS EVALUATION RESULTS OF TRACE SAMPLING

Datasets	Metrics	OCV	STV	TraceCRL	STEAM	iTCRL
TT	Coverage	0.5965	0.8596	0.7018	0.8421	**0.8947**
	Entropy	2.1300	5.1744	2.8690	**5.2043**	3.1684
SN	Coverage	**1.0000**	0.9000	**1.0000**	**1.0000**	**1.0000**
	Entropy	2.6592	2.6007	2.9623	**3.0724**	3.0347
OB	Coverage	**1.0000**	0.9167	**1.0000**	**1.0000**	**1.0000**
	Entropy	2.0335	2.6403	2.2202	2.9819	**3.0905**
Average	Coverage	0.8655	0.8921	0.9006	0.9474	**0.9649**
	Entropy	2.2742	3.4718	2.6838	**3.7529**	3.0979

baselines on three datasets in terms of Coverage. This is because it considers service events that denotes meaningful, singular point in time during the service's duration and learn trace representations from causal perspective, which makes trace more distinguishable. In terms of Entropy, iTCRL is best on OB dataset while ranking third and second on TT dataset and SN dataset, respectively. The reason is that the number of different types of traces is unbalanced in TT dataset and SN dataset, which makes a finer-grained trace representation mean higher Coverage and lower Entropy. Comprehensively considering the effectiveness of all approaches on the three datasets, iTCRL is about 1.85% better than the best baseline (STEAM) in terms of Coverage while ranking third in terms of Entropy.

F. RQ5: Robustness of iTCRL

To evaluate the robustness of iTCRL, we add noise to 20%, 40% and 60% of traces in datasets and conduct comparison experiments. Experimental results on TT dataset, SN dataset and OB dataset are shown in Figs. 23–25, respectively.

As shown in Figs. 23–25, iTCRL changes a little with noise ratio on three datasets in all evaluation metrics. On TT dataset, the change of iTCRL with noise ratio is smaller than all baselines in terms of Macro-F1, F1-score, Coverage and Entropy while slightly larger than STEAM in terms of Micro-F1. On SN dataset, the change of iTCRL with noise ratio is smaller than all baselines in all evaluation metrics. On OB dataset, the

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:31 UTC from IEEE Xplore. Restrictions apply.

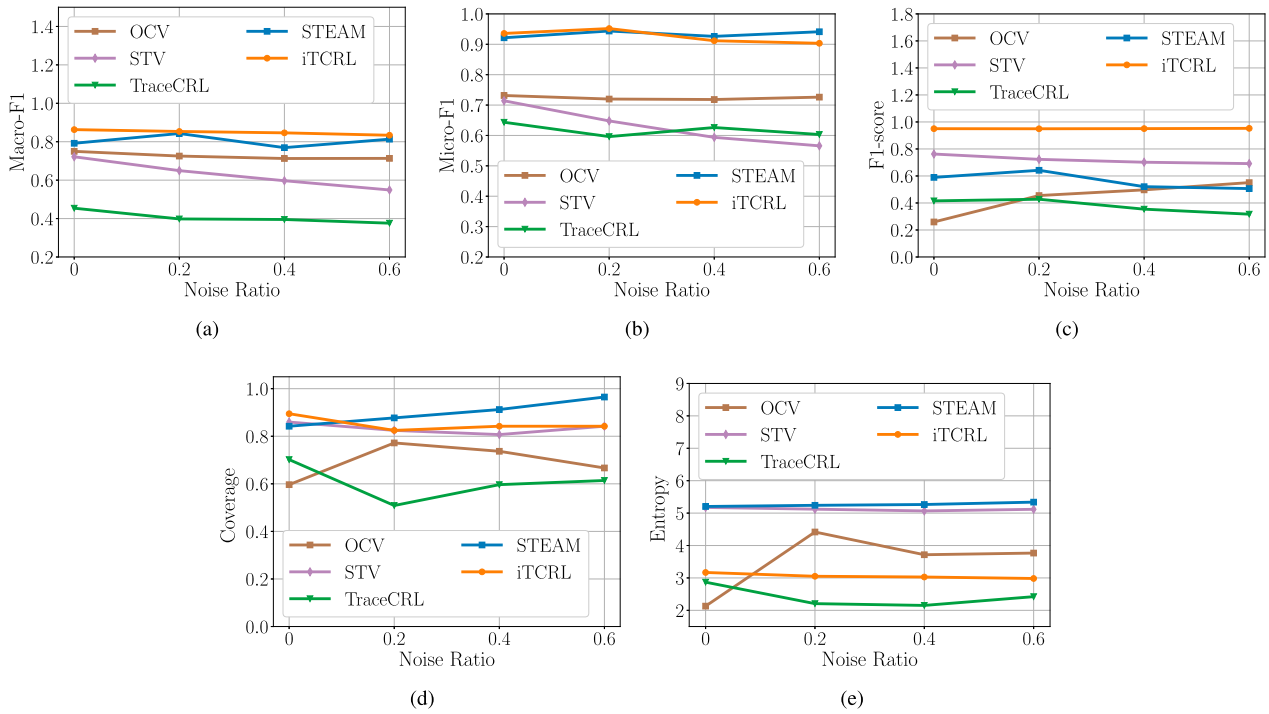


Fig. 23. Robustness of iTCRL on TT dataset.

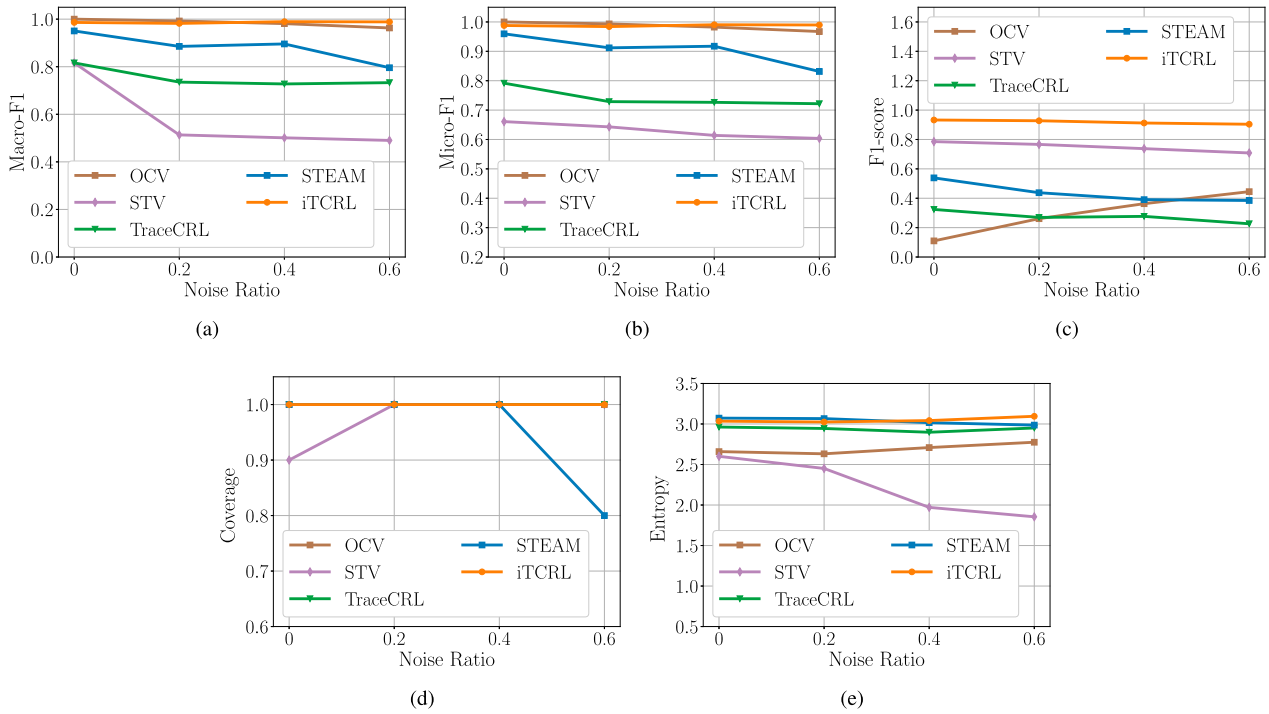


Fig. 24. Robustness of iTCRL on SN dataset.

change of iTCRL with noise ratio is smaller than all baselines in terms of F1-score and Coverage while slightly larger than STEAM in terms of Macro-F1, Micro-F1 and Entropy. The reason is that iTCRL learns trace representations from causal perspective and noise data cannot change traces' causal relationships.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:31 UTC from IEEE Xplore. Restrictions apply.

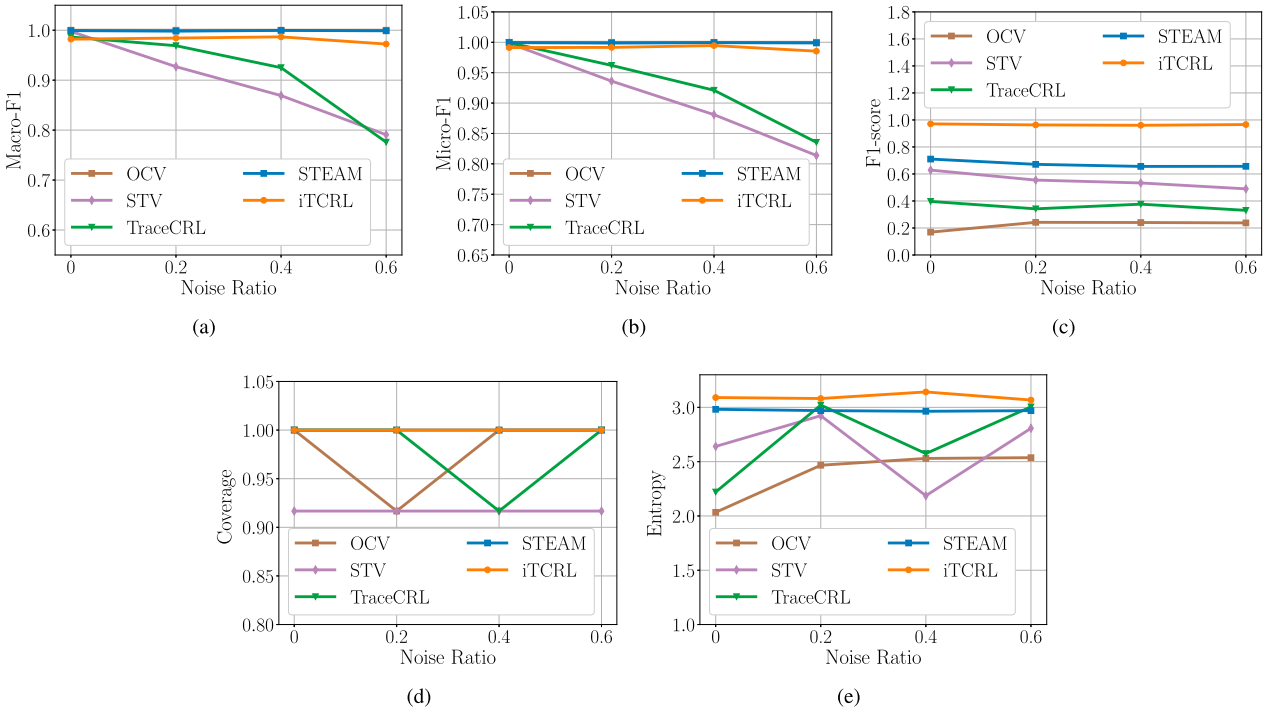


Fig. 25. Robustness of iTCRL on OB dataset.

G. RQ6: Contribution of iTCL

To evaluate the contribution of iTCL, iTCRL is compared with its three variants SETG-HTE-GraphCL, SETG-HTE-JOAO and SETG-HTE-AutoGCL. Experimental results are given in Table VII.

As shown in Table VII, iTCRL outperforms all baselines when comprehensively considering three evaluation tasks. On TT dataset and OB dataset, iTCRL outperforms all baselines in terms of Macro-F1, Micro-F1, F1-score and Coverage while ranking second in terms of Entropy. On SN dataset, iTCRL outperforms all baselines in all evaluation metrics. The reason is that our proposed approach learns trace representations from causal perspective, which avoid the creation of traces that do not exist in real microservice systems.

H. Threats to Validity

For the threats to internal validity, it mainly comes from the implementation of baselines. In these baselines, GraphCL, JOAO and AutoGCL provide open-source implementations [16], [17], [18]. Therefore, we directly use them. For other baselines, we can only develop them based on their papers. To alleviate this threat, we use same libraries to implement them strictly in accordance with its paper and apply the best configuration of hyperparameters to conduct experiments.

For the threats to external validity, it mainly comes from the generalization ability of iTCRL. In this paper, we use experimental datasets from the Train Ticket microservice system, the Social Network microservice system and the Online Boutique microservice system, respectively. Although they are the most commonly used benchmark microservice systems, their scale is still smaller than industrial microservice systems. Therefore, it is yet unknown whether our evaluation results can be generalized to industrial microservice systems. We alleviate this threat from two aspects. First, all fault cases are typical faults in industrial microservice systems. Second, the size of collected traces is comparable to industrial systems.

V. Related Works

In this section, we focus on the related work on graph neural networks, causal inference, graph contrastive learning and trace representation.

Graph Neural Networks. Graph Neural Networks aim to learn a low-dimensional vector representation for nodes or graphs. Bruna et al. [33] first extend convolutions to graph data by graph Laplacian matrix. ChebNet [34] uses a polynomial filter to solve the efficiency problem of spectral-based GNNs. Kipf et al. [35] improve ChebNet by simplifying its parameters to alleviate the overfitting problem. Huang et al. [36] first introduce random filters into graph convolution neural networks to achieve the fast build of graph convolutional network model. Hamilton et al. [37] first propose spatial-based graph neural network framework, which effectively improve generalization to unseen data. Velickovic et al. [38] first introduce the attention mechanism into graph neural networks to reflect the importance of different neighbors. Fu et al. [39] extend graph neural networks from homogeneous graphs to heterogeneous graphs by metapaths. Li et al. [40] design PEGFAN to achieve the accurate node classification in heterophilous graphs. Mo et al. [23] propose a heterogeneous edge-enhanced graph attention network to solve the multi-agent trajectory prediction problem. Li et al. [41] propose a multimodal graph learning approach

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:31 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
CONTRIBUTIONS OF iTCL

Datasets	Metrics	SETG-HTE-GraphCL	SETG-HTE-JOAO	SETG-HTE-AutoGCL	iTCRL
TT	Macro-F1	0.6713	0.7878	0.6742	**0.8630**
	Micro-F1	0.8869	0.8981	0.8464	**0.9357**
	F1-score	0.9486	0.9497	0.9484	**0.9501**
	Coverage	0.7368	0.6491	0.7895	**0.8947**
	Entropy	3.1540	3.3761	**3.6191**	3.1684
SN	Macro-F1	0.9832	0.9703	0.8508	**0.9862**
	Micro-F1	0.9842	0.9696	0.8637	**0.9879**
	F1-score	0.9106	0.8916	0.9158	**0.9326**
	Coverage	**1.0000**	**1.0000**	**1.0000**	**1.0000**
	Entropy	2.9666	2.9794	3.0129	**3.0347**
OB	Macro-F1	0.9495	0.7962	0.8501	**0.9821**
	Micro-F1	0.9538	0.7837	0.8961	**0.9913**
	F1-score	0.9645	0.9657	0.8190	**0.9710**
	Coverage	**1.0000**	**1.0000**	**1.0000**	**1.0000**
	Entropy	**3.0924**	3.0881	3.0222	3.0905
Average	Macro-F1	0.8680	0.8514	0.7917	**0.9438**
	Micro-F1	0.9416	0.8838	0.8687	**0.9716**
	F1-score	0.9412	0.9357	0.8944	**0.9512**
	Coverage	0.9123	0.8830	0.9298	**0.9649**
	Entropy	3.0710	3.1479	**3.2181**	3.0979

to achieve the accurate student engagement prediction. These research about graph neural networks provide theoretical basis for microservice trace representation learning.

Causal Inference. Causal Inference aims to find the causal relationships between things. At present, it has been applied in many domains. For example, Gan et al. [1] use Causal Bayesian Networks to capture the causal relationships between microservices, and counterfactual theory to analyze the root causes of microservice systems. Ma et al. [2] apply causal inference to extract causal relationships between microservices and identify the root causes of microservice system anomalies. Zhang et al. [42] formulate a notion of counterfactual memorization, and apply it to describe the predictions change when a particular document is missing in NLP. These research about causal inference provide theoretical basis for microservice trace representation learning.

Graph Contrastive Learning. Existing graph contrastive learning approaches usually learns graph representation vectors by maximizing the agreement between two contrastive views of the same graph. You et al. [17] design four types of graph augmentation strategies and propose a graph contrastive learning framework based on them, which can produce graph representations of high generalization, transferability and robustness. The same authors [18] propose a unified bi-level optimization framework for GraphCL [17] to select data augmentation strategies automatically, adaptively and dynamically, which can effectively improve the general applicability of GraphCL. Li et al. [43] propose a graph contrastive learning framework for recommender systems. It constructs positive/negative pairs from two simple graph augmentations, which can help recommendation in causes of complex topology or sparse labels. Yin et al. [16] propose an automated graph contrastive learning framework. It uses joint training strategy to learn graph view generators, which can dynamically suit input data. However, the above approaches often create traces that do not exist in real world when they are directly applied to trace representation learning.

Trace Representation. Existing trace representation approaches can be divided into the rule-based approach and the GNN-based approach. The first usually uses the artificial defined features to represent traces. This type of approaches is very easy to implement and has good interpretability. For example, Operation Count Vector (OCV) [32] uses service invocation operations and their number to represent traces. Service Trace Vector (STV) [9] uses service invocation paths and their response time to represent traces. The second usually use machine learning based methods to learn trace representations. This type of approaches can well use the structure information and attribute information of traces. For example, Zhang et al. [27] propose a trace representation approach based on graph neural network and contrastive learning, which can effectively use the graph structured information of traces. He et al. [31] propose a trace representation approach based on graph neural network and the domain knowledge of trace comparison. It incorporates graph neural network and domain knowledge by defining logical clauses, which can avoid the generation of traces that do

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:31 UTC from IEEE Xplore. Restrictions apply.

not exist in real world. However, the above approaches require well-designed graph neural network models and unsupervised learning strategies.

VI. Conclusion and Future Work

In summary, we propose a trace contrastive representation learning approach based on causal intervention. This approach first constructs a Service-Event-based Trace Graph for each trace to describe the runtime status of service events in traces and the complex relationships between them. Then, Causal-intervention-based Trace Contrastive Learning is proposed to learn trace representations from causal perspective, which can effectively aggregate the structure information and attribute information on Service-Event-based Trace Graphs. Finally, we evaluate the effectiveness of iTCRL, robustness of iTCRL and contribution of iTCL with three tasks. Experimental results on three datasets show iTCRL outperforms all baselines in terms of trace classification, trace anomaly detection, trace sampling and noise robustness, and also validate the contribution of iTCL.

Limitations. We note that although our proposed approach achieves best effectiveness in large-scale microservice systems and best comprehensive results on all datasets, its effectiveness in small-scale microservice systems is slightly worse than the best baselines in terms of trace classification. This suggests that our proposed approach more suitable for large-scale microservice systems, and further improvements are needed to adapt it for various scale of microservice systems.

Future Work. Several paths for future researches exist. First, a more fine-grained generation method of causal contrastive views needs to be designed based on the background of microservice systems. This improvement is crucial for adapting the proposed approach to various scale of microservice systems. Second, it is essential to enhance the interpretability of the model. This improvement has far-reaching implications on enhancing credibility for various downstream tasks, such anomaly detection and root analysis.

References

[1] Y. Gan, M. Liang, S. Dev, D. Lo, and C. Delimitrou, "Sage: Practical and scalable ml-driven performance debugging in microservices," in *Proc. 26th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS),* New York, NY, USA: ACM, 2021, pp. 135–151.

[2] M. Ma, W. Lin, D. Pan, and P. Wang, "ServiceRank: Root cause identification of anomaly in large-scale microservice architectures," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 5, pp. 3087–3100, Sep./Oct. 2022.

[3] A. Sriraman and T. F. Wenisch, "μ suite: A benchmark suite for microservices," in *Proc. IEEE Int. Symp. Workload Characterization (IISWC)*, 2018, pp. 1–12.

[4] L. Suresh, P. Bodik, I. Menache, M. Canini, and F. Ciucu, "Distributed resource management across process boundaries," in *Proc. Symp. Cloud Comput. (SoCC),* New York, NY, USA: ACM, 2017, pp. 611–623.

[5] S. Nedelkoski, J. Cardoso, and O. Kao, "Anomaly detection from system tracing data using multimodal deep learning," in *Proc. IEEE 12th Int. Conf. Cloud Comput. (CLOUD),* Piscataway, NJ, USA: IEEE Press, 2019, pp. 179–186.

[6] G. Yu et al., "MicroRank: End-to-end latency issue localization with extended spectrum analysis in microservice environments," in *Proc. Web Conf. (WWW)*, New York, NY, USA: ACM, 2021, pp. 3087–3098.

[7] S. Luo et al., "Characterizing microservice dependency and performance: Alibaba trace analysis," in *Proc. ACM Symp. Cloud Comput. (SoCC)*, New York, NY, USA: ACM, 2021, pp. 412–426.

[8] "Opentelemetry," OpenTelemetry.io. 2024. Accessed: Mar. 27, 2024. [Online]. Available: https://opentelemetry.io/

[9] P. Liu et al., "Unsupervised detection of microservice trace anomalies through service-level deep Bayesian networks," in *Proc. IEEE 31st Int. Symp. Softw. Rel. Eng. (ISSRE)*, 2020, pp. 48–58.

[10] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, 2017, pp. 33–40.

[11] C. Zhang et al., "DeepTraLog: Trace-log combined microservice anomaly detection through graph-based deep learning," in *Proc. 44th Int. Conf. Softw. Eng. (ICSE)*, New York, NY, USA: ACM, 2022, pp. 623–634.

[12] X. Zhang et al., "Robust log-based anomaly detection on unstable log data," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, New York, NY, USA: ACM, 2019, pp. 807–817.

[13] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2014, pp. 1532–1543.

[14] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988.

[15] L. Bai et al., "HAQJSK: Hierarchical-aligned quantum Jensen-Shannon kernels for graph classification," *IEEE Trans. Knowl. Data Eng.*, early access, Apr. 16, 2024, doi: 10.1109/TKDE.2024.3389966.

[16] Y. Yin, Q. Wang, S. Huang, H. Xiong, and X. Zhang, "AutoGCL: Automated graph contrastive learning via learnable view generators," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, no. 8, 2022, pp. 8892–8900.

[17] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen, "Graph contrastive learning with augmentations," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 5812–5823.

[18] Y. You, T. Chen, Y. Shen, and Z. Wang, "Graph contrastive learning automated," in *Proc. Int. Conf. Mach. Learn.,* PMLR, 2021, pp. 12121–12132.

[19] L. Yao, Z. Chu, S. Li, Y. Li, J. Gao, and A. Zhang, "A survey on causal inference," *ACM Trans. Knowl. Discov. Data*, vol. 15, no. 5, pp. 1–46, May 2021.

[20] J. Pearl and D. Mackenzie, *The Book of Why: The New Science of Cause and Effect*. New York, NY, USA: Basic Books, 2018.

[21] J. Pearl, M. Glymour, and N. P. Jewell, *Causal Inference in Statistics: A Primer*. Hoboken, NJ, USA: Wiley, 2016.

[22] X. Fu, J. Zhang, Z. Meng, and I. King, "MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding," in *Proc. Web Conf. (WWW)*, New York, NY, USA: ACM, 2020, pp. 2331–2341.

[23] X. Mo, Z. Huang, Y. Xing, and C. Lv, "Multi-agent trajectory prediction with heterogeneous edge-enhanced graph attention network," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 7, pp. 9554–9567, Jul. 2022.

[24] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 9729–9738.

[25] C. Lee, T. Yang, Z. Chen, Y. Su, and M. R. Lyu, "Eadro: An end-to-end troubleshooting framework for microservices on multi-source data," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, 2023, pp. 1750–1762.

[26] G. Yu, P. Chen, Y. Li, H. Chen, X. Li, and Z. Zheng, "Nezha: Interpretable fine-grained root causes analysis for microservices on multi-modal observability data," *ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, ser. ESEC/FSE 2023, New York, NY, USA: ACM, 2023, pp. 553–565.

[27] C. Zhang et al., "TraceCRL: Contrastive representation learning for microservice trace analysis," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2022, pp. 1221–1232.

[28] X. Zhou et al., "Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study," *IEEE Trans. Softw. Eng.*, vol. 47, no. 2, pp. 243–260, Feb. 2021.

[29] Y. Gan et al., "An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems," in *Proc. 24th Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, New York, NY, USA: ACM, 2019, pp. 3–18.

[30] "Online boutique," Google, 2024. Accessed: Mar. 27, 2024. [Online]. Available: https://github.com/GoogleCloudPlatform/microservices-demo

[31] S. He et al., "STEAM: Observability-preserving trace sampling," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, New York, NY, USA: ACM, 2023, pp. 1750–1761.

[32] P. Las-Casas, J. Mace, D. Guedes, and R. Fonseca, "Weighted sampling of execution traces: Capturing more needles and less hay," in *Proc.*

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:31 UTC from IEEE Xplore. Restrictions apply.

ACM Symp. Cloud Comput., (SoCC), New York, NY, USA: ACM, 2018, pp. 326–332.
[33] J. Bruna, W. Zaremba, A. Szlam, and Y. LeCun, "Spectral networks and locally connected networks on graphs," 2013, *arXiv:1312.6203.*
[34] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 29, D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, Eds., Barcelona, Spain: Curran Associates, Inc., 2016.
[35] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, 2016, pp. 2713–2726.
[36] C. Huang, M. Li, F. Cao, H. Fujita, Z. Li, and X. Wu, "Are graph convolutional networks with random weights feasible?" *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 3, pp. 2751–2768, Mar. 2023.
[37] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., Long Beach, USA: Curran Associates, Inc., 2017.
[38] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Int. Conf. Learn. Representations*, 2018.
[39] X. Wang et al., "Heterogeneous graph attention network," in *Proc. World Wide Web Conf., (WWW),* New York, NY, USA: ACM, 2019, p. 2022–2032.
[40] J. Li, R. Zheng, H. Feng, M. Li, and X. Zhuang, "Permutation equivariant graph framelets for heterophilous graph learning," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Mar. 14, 2004, doi: 10.1109/TNNLS.2024.3370918.
[41] M. Li, X. Zhuang, L. Bai, and W. Ding, "Multimodal graph learning based on 3D Haar semi-tight framelet for student engagement prediction," *Inf. Fusion*, vol. 105, 2024, Art. no. 102224.
[42] C. Zhang, D. Ippolito, K. Lee, M. Jagielski, F. Tramer, and N. Carlini, "Counterfactual memorization in neural language models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., Curran Associates, Inc., 2023, pp. 39321–39362.
[43] M. Li, L. Zhang, L. Cui, L. Bai, Z. Li, and X. Wu, "BLoG: Bootstrapped graph representation learning with local and global regularization for recommendation," *Pattern Recognit.*, vol. 144, 2023, Art. no. 109874.



Xiangbo Tian is currently working toward the Ph.D. degree with the School of Computer Science, Wuhan University, Wuhan, China. His research interests include microservices and AIOps.



Shi Ying is a Professor with the School of Computer Science, Wuhan University, Wuhan, China. His research interests include software engineering, intelligent operation and maintenance management, cloud computing and cloud service software, and artificial intelligence.



Tiangang Li is currently working toward the Ph.D. degree with the School of Computer Science, Wuhan University, Wuhan, China. His research interests include cloud computing, software engineering, resource management, and reinforcement learning.



Mengting Yuan is a Professor with the School of Computer Science, Wuhan University. His research interests include programming languages, compiler technology, and software engineering.



Ruijin Wang is an Associate Professor with the School of Information and Software Engineering, University of Electronic Science and Technology of China, Chengdu, China. He is a Senior Member of the China Computer Federation, a member of the Asia-Pacific Artificial Intelligence Association, and the Secretary-General of the Chengdu Branch of the International Computer Society. Additionally, he is a Visiting Scholar with the Northwestern University, United States and a high-level overseas talent in Sichuan Province. His research interests include cloud-edge intelligent computing, data processing, artificial intelligence, information security, and blockchain.



Yishi Zhao received the M.Sc. and Ph.D. (Distinction) degrees in computing science from Newcastle University, in 2006 and 2011, respectively. He is a Lecturer with the School of Computer Science, China University of Geosciences, Wuhan, China. His research interests include software performance engineering, intelligent decision-making, and applications of blockchain and digital twin.



Jianga Shang (Member, IEEE) received the Ph.D. degree in computer system architecture from Huazhong University of Science and Technology. He is a Professor with the School of Computer Science and a Senior Researcher with the National Engineering Research Center for Geographic Information System, China University of Geosciences, Wuhan. He is also the Director of Human–Cyber–Physical Software for Smart Spaces Lab with the CUG and a member of the ACM, Special Committee on Software Engineering with China Computer Federation. His research interests include mobile and pervasive computing, software technology for human–cyber–physical systems, positioning and navigation of pedestrian, and robotics for smart spaces.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on March 20,2026 at 23:38:31 UTC from IEEE Xplore. Restrictions apply.